

LEWG `[[nodiscard]]` policy

Document number: P3162R0

Date: 2024-02-22

Authors: Darius Neațu <dariusn@adobe.com (mailto:dariusn@adobe.com)>, David Sankel <dsankel@adobe.com (mailto:dsankel@adobe.com)>

Audience: Library Evolution

Abstract

The committee spends substantial time deciding whether or not `[[nodiscard]]` is appropriate for newly introduced standard functions, and the decisions made are often incoherent. We propose a policy that results in minimal syntactic overhead while retaining the detection of the most egregious defects.

Examples illustrating proposed policy

```
// 'empty' is frequently confused with 'clear' by those new to C++.
[[nodiscard]] bool map<T>::empty() const noexcept;

// Discarding the return value of 'new' is always a memory leak.
[[nodiscard]] void* operator new(size_t);

// Discarding the return value of 'operator==' is not a frequent bug
// nor sufficiently catastrophic to justify [[nodiscard]].
template<class T, size_t N>
constexpr bool operator==(const array<T, N>& x, const array<T, N>& y);

// Discarding an 'expected' return value hides errors and marking the class
// [[nodiscard]] requires little syntactic overhead.
template<class T, class E>
class [[nodiscard]] expected { ... };
```

Introduction

`[[nodiscard]]` was introduced in Andrew Tomazos's *Proposal of `[[unused]]`, `[[nodiscard]]` and `[[fallthrough]]` attributes (P0068R0)* (<https://wg21.link/P0068R0>) as a way to generate compiler warnings when `[[nodiscard]]`-marked functions (or functions with `[[nodiscard]]`-marked

return types) are called without handling their return values. The feature was standardized in C++17 with *Wording for `[[nodiscard]]` attribute (P0189R1)* (<https://wg21.link/P0189R1>).

As a rule, the standard encourages but does not mandate compiler warnings. `[[nodiscard]]` is no exception. This fact, combined with the as-if rule^[1], implies standard library implementers are not bound at all by where the C++ standard uses `[[nodiscard]]` annotations. Despite this implementor freedom, annotations were added to the standard library in C++17 with Nicolai Josuttis's *`[[nodiscard]]` in the Library (P0600R1)* (<https://wg21.link/P0600R1>). Josuttis proposed `[[nodiscard]]` placement as follows:

- For existing APIs:
 - not using the return value always is a “huge mistake” (e.g., always resulting in resource leak)
 - not using the return value is a source of trouble and easily can happen (not obvious that something is wrong)
- For new APIs (not been in the C++ standard yet):
 - not using the return value is usually an error.

Following P0600R1's incorporation, the committee has not consistently followed Josuttis's recommendation for new APIs. Without an official policy, each new function's `[[nodiscard]]` annotation was subject to a separate debate, resulting in the present standard's `[[nodiscard]]` incoherence^[2] and time-consuming LEWG discussions^[3].

To resolve inconsistencies and reduce committee debate time, we advocate for the following LEWG policy:

1. Place `[[nodiscard]]` on functions where ignoring a return value is inevitably a severe defect, such as resource leakage.
2. Place `[[nodiscard]]` on functions where overlooking the return value is a common mistake, such as function names frequently confused with others.
3. Place `[[nodiscard]]` on types designed to communicate errors as function return values.

This paper discusses the principles driving this policy, surveys existing practices, considers alternatives, and proposes wording for SD-9.

Principles

No `[[nodiscard]]` policy can address all concerns, so we propose guiding principles to aid decision-making: minimize complexity, focus on the 90% use case, and center on outcomes.

Minimize complexity

Minimizing C++'s complexity makes it more approachable for new users and reduces the maintenance burden of written code, thus improving its longevity. This principle rules out placing `[[nodiscard]]` on almost all functions with non-void return types, even if ignoring these return values is likely a bug.

Focus on the 90% use case

Focusing on the 90% use case recognizes that concentrating on most people's problems rather than all people's problems will generally produce better outcomes, especially when simplicity is at stake. In this case, a handful of `[[nodiscard]]`'s placements is sufficient to generate warnings for the most common and severe bugs.

Consider these examples:

```
std::vector<int> v{...};  
v.empty();           // Using 'clear' instead of 'empty' is a  
                     // common bug, especially for those coming from  
                     // another language.  
  
std::unique_ptr<X> x{...};  
x.release();         // Releasing the 'unique_ptr' in this example  
                     // results in a memory leak.  
  
std::async(job_x, &x, ...); // Accidentally ignoring the return value of  
std::async(job_y, &y, ...); // async gives the false impression that jobs  
                           // are run in parallel.  
  
calloc(size * sizeof(int)); // Ignoring the return value of calloc is  
                           // a memory leak.
```

Center on outcomes

Centering on outcomes is about considering the practical impact of a decision. What impact will a `[[nodiscard]]` policy have? We've identified two:

1. **Standard library implementations.** Implementors are not obligated to follow `[[nodiscard]]` guidance. `libstdc++` and Visual C++ make their own decisions^[4], but `libc++` mimics the standard's placement^[5]. The impact here is minimal.
2. **Training content.** Training, books, and websites, including the incredibly popular `cppreference.com` (<https://cppreference.com/>), frequently reproduce function signatures in the standard library. These signatures, as a result, heavily influence coding style.

Whether or not the standard is *intended* to impact coding style, *it does* and the committee should pay attention to this impact. Do we want the standard to encourage, by example, a conservative placement of `[[nodiscard]]` annotations, placement of `[[nodiscard]]` almost everywhere, or to avoid using the feature entirely? It is our position that users are best served by the former.

Survey of existing practice

This section surveys existing `[[nodiscard]]` placement in the standard library, tooling, and the C++ Core guidelines. In short, the standard lacks consistency, tooling tends towards warning on all unused returns, and the C++ Core Guidelines have little to say on the subject. It is no wonder that users are frequently perplexed about what to do with the feature^[6].

The standard library

The code snippets below show various signatures that either get or do not get the `[[nodiscard]]` attribute in the December 2023 C++ Working Draft (N4971) (<https://wg21.link/N4971>)

- `.empty()` (~30 occurrences)

```
// containers deque, list, map, ... , unordered_map, unordered_set
[[nodiscard]] bool map<T>::empty() const noexcept;
// utilities: basic_stacktrace, path, ...
[[nodiscard]] bool path<T>::empty() const noexcept;
// string types: basic_string_view, basic_string
[[nodiscard]] constexpr bool basic_string_view<T>::empty() const noexcept;
```

This addresses `.empty()`'s confusion with `.clear()`. All `.empty()` instances were marked as `[[nodiscard]]`.

- Allocation functions (~20 occurrences)

```
// C++ allocation functions: marked as [[nodiscard]]
// * operator new overloads:
[[nodiscard]] void* operator new(size_t);
[[nodiscard]] void* operator new[](size_t);
// * allocate() overloads
[[nodiscard]] constexpr T* allocator<T>::allocate(size_t n);
[[nodiscard]] void* pmr::allocate(size_t bytes, size_t alignment = max_align);
```

Discarding a value returned by an allocation function is undoubtedly a memory leak.

- `async` (2 occurrences)

```
// async overloads
template<class F, class... Args>
[[nodiscard]] future<...> async(F&& f, Args&&... args);
```

Not capturing the return value for `async` results in surprising synchronous execution.

- Standard error types (0 occurrences) -

```
template<class T, class E>
class expected { ... };
template<class E>
class unexpected { ... };

class error_code { ... };
class error_condition { ... };
enum class errc { ... };
```

None of the standard error types have a `[[nodiscard]]` attribute.

- Inconsistencies

```

// allocation functions
// * default: do have [[nodiscard]]
[[nodiscard]] void* operator new(size_t);
// * exceptions: C allocation functions
void* malloc(size_t size);
void* calloc(size_t nmemb, size_t size);
void* realloc(void* ptr, size_t size);

// operator==
// * default: does not have [[nodiscard]]
constexpr bool operator==(const vector<...>& x, const vector<...>& y);
constexpr bool operator==(const basic_string<...>& lhs, const basic_string<...>& r
// * exceptions: few new APIs
[[nodiscard]] friend bool operator==(const stop_token& lhs, const stop_token& rhs)
[[nodiscard]] friend bool operator==(const stop_source& lhs, const stop_source& r

// get_id() for threads
// * (old) free function API
id this_thread::get_id() noexcept;
// * (new) member function API
[[nodiscard]] id jthread::get_id() const noexcept;

```

Tooling: Clang Tidy

Clang Tidy provides the `modernize-use-nodiscard` (<https://clang.llvm.org/extra/clang-tidy/checks/modernize/use-nodiscard.html>) flag, which suggests and automatically applies `[[nodiscard]]` to a codebase in specific scenarios, such as non-void non-template return const member function.

```

bool class_name::empty() const;           // warning: should be marked [[nodiscard]]
bool class_name::check(int i) const; // warning: should be marked [[nodiscard]]

```

Moreover, the experimental flag `bugprone-unused-return-value` (<https://clang.llvm.org/extra/clang-tidy/checks/bugprone/unused-return-value.html>) generates warnings when a user ignores the return value of specific Standard Library functions. An example is `std::unique_ptr::release()`, which may be confused with `std::unique_ptr::reset()`.

Similarly to our proposed policy, this tool warns when standard error type return values are not handled:

```
// warning: the value returned by this function should be used
std::expected<int, int> foo() { return {1}; }

void bar() { foo(); }
```

The C++ Core Guidelines

A recent GitHub issue in the C++ Core Guidelines

(<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines>) repository, `[[nodiscard]]` advice is largely absent (<https://github.com/isocpp/CppCoreGuidelines/issues/2109>), points out that the C++ Core Guidelines does not mention `[[nodiscard]]` aside from not ignoring results of functions with this attribute. Considering section SL.2 ("SL.2: Prefer the standard library to other libraries" (<https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rsl-sl>)), the missing guideline from C++ Core Guidelines should be consistent with a LEWG policy.

The discussion led to multiple possible guidelines that are still under debate:

- A: Add `[[nodiscard]]` when it disambiguates what a function does or discarding the result is a safety issue.
- B: Add `[[nodiscard]]` when it is likely to avoid the misuse of a function.
- C: Add `[[nodiscard]]` when a function is impossible to use when the caller discards the result.
- D: Add `[[nodiscard]]` when discarding the result of a function with side effects is not intended.

The guideline *A* uses the attribute only for critical situations (e.g., `.empty()`). In the opposite direction, the *B* version suggests to put `[[nodiscard]]` almost everywhere (e.g., even on `operator==`). The *C* wording limits the usage to specific cases where code may break (e.g., `std::launder`, `std::async`), while *D* aims to use `[[nodiscard]]` only on functions with side-effects, such `read()`.

Alternatives considered

1. **Place `[[nodiscard]]` on every function where discarding the result is likely a bug.**
Because this would result in `[[nodiscard]]` placement on almost every non-void returning function, it goes against the principle of minimizing complexity.
2. **Do not use `[[nodiscard]]` in the standard library.** This alternative goes against the principle of centering on outcomes. If people do not see `[[nodiscard]]` used in the standard library or relevant documentation (such as `cppreference`), then the positive

impact the feature can have is diminished.

Wording for SD-9

Append to List of Standard Library Policies (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2267r1.html#list-of-standard-library-policies>) section of SD-9 (P2267R1: Library Evolution Policies (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2267r1.html>)):

X. **[[nodiscard]]** policy:

X.1. (P3162R0) Place **[[nodiscard]]** on functions where ignoring a return value is inevitably a severe defect, such as resource leakage.

X.2. (P3162R0) Place **[[nodiscard]]** on functions where overlooking the return value is a common mistake, such as function names frequently confused with others.

X.3. (P3162R0) Place **[[nodiscard]]** on types designed to communicate errors as function return values.

Conclusion

This paper demonstrated the need for a policy regulating **[[nodiscard]]** use in the standard library and proposed such a policy. It outlined the principles motivating this policy, analyzed existing practices, and considered alternatives.

References

- Christopher Di Bella. P2377R0: **[[nodiscard]]** in the Standard Library: Clause 23 Iterators library. <https://wg21.link/P2377R0> (<https://wg21.link/P2377R0>), 05/2021
- Hana Dusíková. P2351R0: Mark all library static cast wrappers as **[[nodiscard]]**. <https://wg21.link/P2351R0> (<https://wg21.link/P2351R0>), 04/2020
- Nicolai Josuttis. P0600R1: **[[nodiscard]]** in the Library, Rev1. <https://wg21.link/p0600r1> (<https://wg21.link/p0600r1>), 11/2017.
- Inbal Levi et al. P2267R1: Library Evolution Policies.
- Andrew Tomazos. P0189R1: Wording for **[[nodiscard]]** attribute. <https://wg21.link/p0189r1> (<https://wg21.link/p0189r1>), 2/2016.
- Andrew Tomazos. P0068R0: Proposal of **[[unused]]**, **[[nodiscard]]** and **[[fallthrough]]** attributes. <https://wg21.link/p0068r0> (<https://wg21.link/p0068r0>), 9/2015.
- Daniel Towner, Ruslan Arutyunyan. P2993R1: Extend `<bit>` header function with overloads for `std::simd`. <https://wg21.link/P2933R1> (<https://wg21.link/P2933R1>), 8/2023 <https://wg21.link/P2267R1> (<https://wg21.link/P2267R1>), 11/2023

- Ville Voutilainen. D2422R0: Remove nodiscard annotations from the standard library specification. <https://isocpp.org/files/papers/D2422R0.html> (<https://isocpp.org/files/papers/D2422R0.html>), 08/2021.
1. For details on the "as-if" rule, see paragraph 1 of [intro.abstract] in the December 2023 C++ Working Draft (N4971) (<https://wg21.link/N4971>) and its footnote 
 2. For a recent example, see the 2023 Kona discussion on *[[nodiscard]]* placement for *Extend <bit> header function with overloads for std::simd (P2993R1)* (<https://wg21.link/P2993R1>) 
 3. LEWG related papers: Christopher Di Bella's P2377R0: *[[nodiscard]]* in the Standard Library: Clause 23 Iterators library (<https://wg21.link/P2377R0>), Ville Voutilainen's D2422R0: Remove nodiscard annotations from the standard library specification (<https://isocpp.org/files/papers/D2422R0.html>), Hana Dusíková's P2351R0: Mark all library static cast wrappers as nodiscard (<https://wg21.link/P2351R0>). 
 4. Note libstdc++'s placement (https://github.com/gcc-mirror/gcc/blob/3232ebd91ed55b275b9d5a6e8355336382c4afd5/libstdc%2B%2B-v3/include/bits/stl_vector.h#L1073) of *[[nodiscard]]* on both *vector::capacity* and *vector::empty* where the standard has placement only on the latter. The same is true for Visual C++ (<https://github.com/microsoft/STL/blob/c53ac59abae552e50654943f7f80f23376a8039a/stl/inc/vector#L1866>) 
 5. See *vector::empty()* and *vector::capacity()* inside libc++ (<https://github.com/llvm/llvm-project/blob/2167881f5154823dc3183845700add7df15fc856/libcxx/include/vector#L601>). 
 6. See this stackoverflow discussion (<https://softwareengineering.stackexchange.com/questions/363169/whats-the-reason-for-not-using-c17s-nodiscard-almost-everywhere-in-new-c>) for an example 